

TriageAI Clinical Assistant

Use Case D — AI Clinical Triage System

Project

AI Clinical Triage Assistant

Repository

triage-ai-clinical-assistant

1. Executive Summary

TriageAI is a production-grade AI Clinical Triage Assistant built to assess patient inputs and generate structured triage severity ratings, clinical recommendations, and real-time risk flags. The system was developed as a response to Use Case D, which required structured output, multi-step reasoning, and safety guardrails over a prescribed Input → Extraction → Risk Engine → LLM → Recommendation → Safety Validator pipeline.

The final implementation significantly exceeds the minimum requirements. It delivers a stateful, multi-layered pipeline orchestrated with LangGraph, backed by a 12,127-entry FAISS disease ontology index, a three-tier LLM waterfall strategy for operational resilience, a Human-in-the-Loop clarification mechanism for ambiguous cases, full Pydantic v2 structured output with auto-repair validation, a SQLite persistence layer, and a multi-page Streamlit clinician portal with a separate FastAPI REST backend.

Key Outcome: A fully operational clinical decision support system that never fails silently — every patient input produces a validated, structured TriageResult regardless of LLM provider availability.

Deliverables Status

Codebase	Complete — Python, FastAPI, Streamlit, LangGraph, FAISS, SQLite
Structured Output	Pydantic v2 TriageResult schema with auto-repair validator
Multi-step Reasoning	7-stage LangGraph pipeline with stateful checkpointing
Safety Guardrails	Hard-rule risk engine + HITL loop + safety validator fallback
Architecture	Fully documented — see Architecture Deliverable
Test Cases	Pytest unit tests + 10 clinical benchmark scenarios
Prompts	18 prompts across 6 development phases — see Prompts Deliverable

2. Problem Statement & Requirements

2.1 Original Problem Statement

Use Case D required building an AI Clinical Triage Assistant capable of assessing patient inputs to provide triage severity, recommendations, and risk flags. The prescribed architecture was:

Prescribed Pipeline: Input → Extraction → Risk Engine → LLM → Recommendation → Safety Validator

2.2 Core Requirements

Structured Output	All LLM responses must conform to a validated schema — no freeform text reaching the API caller
Multi-step Reasoning	The system must reason in stages, not in a single LLM call, to prevent hallucination
Safety Guardrails	Dangerous or ambiguous outputs must be caught, repaired, or escalated before delivery

2.3 Identified Failure Modes

The problem statement explicitly listed three failure modes that the system must handle gracefully:

- Ambiguous symptoms — patient descriptions that are too vague for reliable triage
- Missing vitals — incomplete vital sign data that compromises risk assessment
- Conflicting indicators — cases where vitals and symptoms point to different severity levels

3. System Architecture

3.1 High-Level Design

TriageAI is built as a decoupled, stateful graph using LangGraph as the orchestration backbone. The system separates concerns clearly: NLP extraction, deterministic rule evaluation, semantic retrieval, LLM reasoning, human validation, and structured output are all independent, testable stages.

3.2 Processing Pipeline (7 Stages)

#	Stage	Description	Module
01	Clinical NLP Extraction	Normalizes freeform complaint text using ScispaCy NER (with keyword fallback). Extracts symptoms, anatomical locations, comorbidities, medications, and temporal flags. Computes a Data Completeness Score.	pipeline/extractor.py
02	Hard Rule Risk Engine	Evaluates vital signs against standard medical thresholds. Can bypass LLM entirely to fast-track critical cases (e.g., SpO2 < 85%, GCS <= 8) directly to ESI 1 output.	pipeline/risk_engine.py
03	Vector Semantic Search	Queries a FAISS index of 12,127 Human Disease Ontology entries using sentence-transformer embeddings. Returns top-5 disease matches with ICD codes and L2 similarity scores.	rag/retriever.py
04	LLM Waterfall Router	Calls the tiered LLM chain (Groq → MedGemma → Gemini) with the full assembled context prompt. Returns structured JSON for the triage result.	pipeline/llm_router.py
05	HITL Clarification Loop	If completeness score is below threshold or symptoms are ambiguous, LangGraph emits a checkpoint interrupt. The clinician is prompted for 5 targeted questions before the pipeline resumes.	pipeline/langgraph/nodes.py
06	Schema Validation & Repair	Parses LLM output against the Pydantic TriageResult schema. Auto-repairs missing fields with safe clinical defaults. Never returns empty or unvalidated output.	pipeline/safety_validator.py

07	Recommendation Builder	Formats the validated TriageResult into a complete clinical handoff report. Persists the case to SQLite in a background task.	utils/report_generator.py
----	------------------------	---	---

3.3 LLM Waterfall Strategy

To ensure the system always returns a response even under provider outages or rate limits, a three-tier waterfall is implemented. Each tier is attempted sequentially with timeout enforcement:

Tier 1 — Primary	Groq API (llama-3.3-70b-versatile) — fastest inference, called first on every request
Tier 2 — Medical Fallback	HuggingFace (google/medgemma-27b-it) — medically fine-tuned, activated on Groq failure
Tier 3 — Last Resort	Google Gemini 2.5 Flash — guarantees a response is always returned

3.4 Technology Stack

Orchestration	LangGraph — stateful pipeline graph with SQLite checkpointing for HITL
NLP	ScispaCy (en_core_sci_sm) + keyword fallback parser
Vector Search	FAISS (faiss-cpu) with sentence-transformers (all-MiniLM-L6-v2)
Backend	FastAPI + Uvicorn — async REST API with 10 endpoints
Frontend	Streamlit — multi-page clinician portal (4 pages)
Data Validation	Pydantic v2 — strict schema enforcement with field validators
Persistence	SQLite + aiosqlite + SQLAlchemy 2.0 async
Visualization	Plotly — dashboard charts in Streamlit status page
LLM Providers	Groq SDK, HuggingFace Hub, Google GenAI SDK

4. Implementation Details

4.1 Codebase Structure

The repository follows a modular, production-grade layout with clear separation of concerns across 8 top-level directories:

pipeline/	Core triage logic: extractor, risk engine, LLM router, safety validator, LangGraph nodes and graph
rag/	FAISS index builder and retriever singleton for disease ontology search
models/	Pydantic v2 schemas: PatientInput and TriageResult
frontend/	Multi-page Streamlit portal: triage, history, status, about pages
db/	SQLAlchemy async database layer: tables, CRUD operations
utils/	Shared utilities: logger, prompt templates, clinical report generator
tests/	Pytest unit tests and benchmark_cases.json with 10 clinical scenarios
Root	main.py (FastAPI), streamlit_app.py, config.py, config.ini, requirements.txt

4.2 Structured Output Design

All triage output is governed by the TriageResult Pydantic v2 model. The schema enforces clinical correctness at the data layer — no downstream code ever receives a malformed response. Key fields include:

- triage_level — integer 1–5 (ESI scale), validated with field_validator
- confidence_score — float 0.0–1.0, capped based on data completeness
- risk_flags — list of strings, injected from risk engine if LLM omits them
- rag_matches — list of RAGMatch objects with disease name, ICD code, and L2 similarity score
- warnings — auto-populated when auto-repair or fallback output is used
- completeness_score — surfaced to the UI to guide clinicians on missing data

4.3 HITL Clarification Mechanism

When the data completeness score falls below 0.6 or ambiguous keywords are detected in the chief complaint, LangGraph pauses the graph using its built-in interrupt mechanism. The state is checkpointed to SQLite under a unique thread_id. The API returns a pending_clarification response with five targeted clinical questions generated by the LLM. Once the clinician submits answers via POST /api/triage/resume, the graph resumes from the checkpoint with the clarification answers injected into the state, allowing the LLM node to produce a more confident assessment.

4.4 Safety Validator & Auto-Repair

The safety_validator.py module acts as the final gatekeeper before any output reaches the API caller. It implements a three-stage recovery strategy:

- Stage 1: Attempt JSON parse of raw LLM output (handles markdown code fences)
- Stage 2: Validate against Pydantic TriageResult schema
- Stage 3 (if validation fails): Auto-repair — fill missing fields with conservative clinical defaults, inject risk engine flags if LLM omitted them, set confidence to 0.1 to signal uncertainty
- Stage 4 (if repair fails): Construct a minimal safe fallback result with ESI 2 and a mandatory human review flag

Safety Principle: The system will never return an empty or unvalidated response. When in doubt, it defaults to ESI 2 (Emergent) — the conservative clinical choice.

5. API Reference

The FastAPI backend exposes 10 endpoints with async request handling, CORS middleware, request logging, and a global exception handler that never exposes stack traces to API callers.

Meth od	Endpoint	Description
POST	<i>/api/triage/</i>	Start a new LangGraph triage run. Returns complete result or pending clarification.
POST	<i>/api/triage/resume</i>	Resume a paused HITL run with clinician-provided clarification answers.
POST	<i>/api/clarify</i>	Generate 5 targeted clinical questions for a given chief complaint.
GET	<i>/api/cases</i>	Paginated case history with filters: ESI level, model used, date range.
GET	<i>/api/cases/{id}</i>	Retrieve a single case by session ID.
DELETE	<i>/api/cases/{id}</i>	Delete a case from history.
GET	<i>/api/stats</i>	Aggregate statistics: ESI distribution, model usage counts, avg confidence.
GET	<i>/api/health</i>	System health check: DB connection, RAG index readiness, uptime.

6. Test Cases & Validation

6.1 Unit Test Coverage

The tests/ directory contains pytest unit tests covering all core pipeline components. Tests are designed to run without live API keys by mocking LLM calls.

test_extractor.py	Symptom extraction from freeform text, completeness scoring, ScispaCy fallback behavior
test_risk_engine.py	All ESI 1 and ESI 2 hard rule triggers, conflicting indicator handling, keyword matching
test_safety_validator.py	Schema repair on malformed output, conservative fallback assignment, risk flag injection

6.2 Clinical Benchmark Scenarios

Ten clinical benchmark cases (benchmark_cases.json) are defined to validate end-to-end ESI assignment accuracy. These cover the full severity spectrum and all major failure modes:

#	Clinical Scenario	Failure Mode Tested	Expected ESI
1	STEMI — chest pain with diaphoresis and radiation to left arm	Conflicting indicators	ESI 1/2
2	Anaphylaxis — bee sting, hives, throat tightening, stridor	Hard rule fast-track	ESI 1
3	Critical hypoxemia — SpO2 84%, no other symptoms provided	Missing vitals context	ESI 1
4	Moderate asthma exacerbation — wheezing, SpO2 92%, accessory muscles	Normal pipeline	ESI 2/3
5	UTI — burning urination, low-grade fever, no comorbidities	Normal pipeline	ESI 4
6	Minor laceration — 3cm hand cut, no vitals provided	Missing vitals	ESI 4/5
7	Hypoglycemia — diabetic, blood glucose 45 mg/dL, diaphoresis	Conflicting indicators	ESI 2
8	Tension headache — gradual onset, normal vitals, no red flags	Normal pipeline	ESI 4/5
9	Meningitis — fever, neck stiffness, photophobia, petechiae	Ambiguous symptoms	ESI 1/2
10	Ankle sprain — stable vitals, no comorbidities, pain 4/10	Normal pipeline	ESI 4/5

7. Failure Mode Handling

The three prescribed failure modes from the problem statement are addressed by distinct, layered mechanisms. Each failure mode has a detection strategy, a response path, and a user-facing signal.

7.1 Ambiguous Symptoms

Detection: The data completeness score drops below 0.6, or the chief complaint text contains ambiguity keywords ("unclear", "vague", "unknown", "unspecified").

Response: LangGraph emits a checkpoint interrupt, pausing the pipeline. The LLM generates 5 patient-specific clinical clarifying questions, sorted by urgency. The clinician portal displays these questions with clinical rationale. The pipeline resumes only after answers are submitted.

Signal to User: The Streamlit UI shows a "Clarification Required" state with the specific questions highlighted. The confidence score in the final output reflects the additional data provided.

7.2 Missing Vital Signs

Detection: The extractor's completeness scoring weights critical vitals (SpO2, HR, BP) more heavily than optional ones. Missing critical vitals trigger a significant score penalty.

Response: The specific missing vitals are surfaced both in the API response (completeness_score, missing_vitals fields) and in the Streamlit UI. The HITL loop can request the missing vitals explicitly. The safety validator caps confidence_score at 0.6 when vitals are incomplete.

Signal to User: The triage result includes a warnings field listing the missing vitals. The completeness score is displayed as a progress bar in the clinician UI.

7.3 Conflicting Indicators

Detection: The risk engine evaluates hard thresholds independently of the LLM. When vital-based rules fire but symptom-based severity appears lower (or vice versa), the two assessments diverge.

Response: The risk engine's assessment always wins in critical cases — if it triggers fast_track=True, the pipeline bypasses the LLM entirely and assigns the risk engine's ESI level. The safety validator additionally injects any risk engine flags that the LLM may have omitted from its risk_flags output.

Signal to User: Triggered risk engine rules are listed verbatim in the `risk_flags` field of the output, clearly labelled as hard rule violations. The clinician UI displays these in red alert boxes.

8. Strengths, Limitations & Future Work

8.1 Strengths

- Operational resilience — three-tier LLM waterfall ensures the system always returns a response
- Clinical safety-first design — hard rules take precedence over LLM output for critical vitals
- No silent failures — every failure path (LLM timeout, schema error, index miss) has an explicit handling strategy
- HITL mechanism — clinician remains in control for ambiguous cases, not just overridden by AI
- Structured output throughout — Pydantic schemas at every boundary prevent unvalidated data from propagating
- Full audit trail — SQLite persistence and structured logging for every triage transaction
- Production-ready architecture — modular, testable, and deployable with minimal configuration

8.2 Limitations

- MedGemma inference via HuggingFace API can be slow (30–90 seconds) under load, affecting Tier 2 fallback latency
- The FAISS index is built from a static disease ontology snapshot — it does not update dynamically with new medical literature
- ScispaCy's `en_core_sci_sm` model is a small biomedical NER model and may miss domain-specific clinical entities in complex cases
- Benchmark accuracy cannot be formally reported without a live LLM environment — ESI correctness depends on prompt adherence which varies per model
- No authentication layer on the REST API — suitable for assessment/prototype but not production deployment without auth middleware

8.3 Recommended Future Enhancements

- Replace static FAISS index with a continuously updated vector database (Pinecone or Weaviate) fed from PubMed
- Add an authentication and role-based access control layer to the FastAPI backend
- Implement model comparison mode — run all three LLM tiers in parallel and present the consensus result
- Add a feedback loop where attending physicians can flag incorrect ESI assignments to fine-tune the prompts
- Integrate vitals monitoring device APIs for automatic vital sign ingestion
- Add DICOM integration for radiology report analysis as an additional triage input

9. Setup & Execution

Python Version	3.10 or higher required
Primary LLM	Groq (llama-3.3-70b-versatile) — obtain free API key at console.groq.com
Medical LLM	MedGemma-27B via HuggingFace — requires HF_TOKEN with model access approved
Fallback LLM	Gemini 2.5 Flash — obtain API key at aistudio.google.com

Quick Start

- Step 1 — Install dependencies:** `pip install -r requirements.txt`
- Step 2 — Configure environment:** Copy `.env.example` to `.env` and populate API keys
- Step 3 — Start backend:** `uvicorn main:app --host 0.0.0.0 --port 8000 --reload`
- Step 4 — Start frontend:** `streamlit run streamlit_app.py`

The system is accessible at `http://localhost:8501` (Streamlit) and `http://localhost:8000/docs` (FastAPI Swagger UI).

10. Conclusion

TriageAI delivers a complete, professional-grade implementation of the AI Clinical Triage Assistant use case. The system goes beyond the minimum requirements by implementing a stateful multi-step graph, medical-domain semantic retrieval, a human-in-the-loop clarification mechanism, a three-tier LLM waterfall for operational resilience, and a rigorous structured output pipeline with auto-repair validation.

The architecture is clinically grounded — hard rules always supersede LLM judgement for life-threatening vital sign combinations, the system never returns an empty response, and every triage transaction is logged with a full audit trail. The system is best understood not as a replacement for clinical judgement, but as a high-speed, protocol-consistent first pass that surfaces the most clinically relevant information to the treating clinician in a structured, actionable format.

Disclaimer: TriageAI is an AI-generated clinical decision support prototype. All triage outputs must be reviewed and confirmed by a licensed physician before clinical action is taken.